

R2.09 — Initiation au développement Web

Durée : 2h · Travail en binôme · Pré-requis : TD2 terminé (auth Supabase fonctionnelle)

Objectifs du TD

Modèle BDD complet	Créer les 3 tables (boards, categories, tasks) dans Supabase
Données de test	Insérer des catégories et une première tâche via l'éditeur SQL
Composant TaskCard	Afficher une carte de tâche avec titre, priorité, catégorie, deadline
Liste de tâches	Charger et afficher les tâches depuis Supabase dans le dashboard
Formulaire de création	Permettre l'ajout d'une tâche via un formulaire React

Partie A — Récapitulatif & vérification (10 min)

Rappel TD2

Votre app doit : afficher la page /login → permettre la création de compte et la connexion → rediriger vers /dashboard → lister les profils utilisateurs depuis la table profiles.

A.1 Vérification rapide

1. npm run dev → connexion/déconnexion fonctionne.
2. Le dashboard affiche bien le tableau des profils.
3. git status est propre (tout est commité).

A.2 Rappel — Architecture du projet

À l'issue de ce TD, votre arborescence src/ sera :

src/

lib/

supabase.js

pages/

Partie B — Modèle de données dans Supabase (30 min)

Dans ce TD, vous allez créer les trois tables du projet KanbanRT telles que définies dans le cours magistral. Chaque table correspond à une entité métier.

B.1 Table categories

Allez dans Table Editor → New table → Nom : categories. Ajoutez les colonnes suivantes :

Colonne	Type	Contrainte	Description
id	uuid	gen_random_uuid()	Clé primaire
name	text	UNIQUE, NOT NULL	Nom de la catégorie
color	text	—	Couleur hexadécimale (#1A8C82)



La contrainte UNIQUE sur name

En cochant 'Is Unique' sur la colonne name, vous garantissez qu'il ne peut pas exister deux catégories avec le même nom. Supabase renverra une erreur 409 si vous tentez un doublon.

B.2 Table boards

Créez la table boards (tableau Kanban) avec les colonnes suivantes :

Colonne	Type	Contrainte	Description
id	uuid	gen_random_uuid()	Clé primaire
title	text	NOT NULL	Nom du tableau
owner_id	uuid	FK → auth.users.id	Propriétaire du tableau
created_at	timestampz	now()	Date de création automatique



Clé étrangère owner_id

Pour créer la FK vers auth.users.id : dans la colonne owner_id, cliquez sur 'Foreign Key', choisissez le schéma auth, la table users, et la colonne id. Supabase gère l'intégrité référentielle automatiquement.

B.3 Table tasks — Table principale

C'est la table centrale de l'application. Créez-la avec toutes les colonnes ci-dessous :

Colonne	Type	Valeur par défaut / Contrainte	Description
id	uuid	gen_random_uuid()	Clé primaire, générée auto
title	text	NOT NULL	Titre de la tâche (obligatoire)
description	text	—	Description détaillée
status	text	'todo'	todo in_progress review done
priority	text	'low'	low medium high
due_date	date	—	Date d'échéance
created_at	timestampz	now()	Horodatage automatique
category_id	uuid	FK → categories.id	Catégorie optionnelle
created_by	uuid	FK → auth.users.id	Créateur de la tâche
board_id	uuid	FK → boards.id	Tableau Kanban associé



Les 4 statuts Kanban

Le champ status est de type text avec une valeur par défaut 'todo'. Les 4 valeurs attendues sont : todo (À faire) | in_progress (En cours) | review (Validation) | done (Terminée). En TD5, on ajoutera une contrainte CHECK pour forcer ces valeurs.

B.4 Insérer des données de test (SQL Editor)

Dans Supabase, allez dans SQL Editor → New query. Copiez et exécutez le script suivant :

```
-- 1. Insérer des catégories
```

```
INSERT INTO categories (name, color) VALUES
```


Vérifiez dans Table Editor → tasks que les 4 tâches sont bien insérées.

Partie C — Composant TaskCard (25 min)

C.1 Créer le composant TaskCard.jsx

Ce composant affiche une carte de tâche avec toutes ses informations. Créez `src/components/TaskCard.jsx` :

```
// src/components/TaskCard.jsx

// Couleurs selon la priorité

const PRIORITY_COLORS = {

  high:    { bg: '#FEF2F2', border: '#DC2626', label: '🔴 Haute' },

  medium: { bg: '#FFFBEA', border: '#F59E0B', label: '🟡 Moyenne' },

  low:     { bg: '#F0FDF4', border: '#16A34A', label: '🟢 Basse' },

};

// Libellés selon le statut

const STATUS_LABELS = {

  todo:      { label: '📋 À faire',    color: '#64748B' },

  in_progress: { label: '⚙️ En cours',  color: '#3B82F6' },

  review:     { label: '👁️ Validation', color: '#F59E0B' },

  done:       { label: '✅ Terminée',  color: '#16A34A' },

};

export default function TaskCard({ task, onDelete }) {

  const priority = PRIORITY_COLORS[task.priority] || PRIORITY_COLORS.low;

  const status   = STATUS_LABELS[task.status]    || STATUS_LABELS.todo;
```



```

// Formater la date d'échéance

const dueLabel = task.due_date

    ? new Date(task.due_date).toLocaleDateString('fr-FR', {
        day: '2-digit', month: 'short', year: 'numeric'
    })

    : null;

// Vérifier si la tâche est en retard

const isOverdue = task.due_date &&

    new Date(task.due_date) < new Date() &&

    task.status !== 'done';

return (

    <div style={{
        background: priority.bg,
        border: `2px solid ${priority.border}`,
        borderRadius: '10px',
        padding: '1rem',
        marginBottom: '0.75rem',
        boxShadow: '0 2px 6px rgba(0,0,0,0.07)',
        transition: 'transform 0.15s',
    }}>

        {/* En-tête : titre + bouton supprimer */}

        <div style={{ display: 'flex', justifyContent: 'space-between',
            alignItems: 'flex-start', gap: '0.5rem' }}>

```



```

</span>

{/* Priorité */}

<span style={{ fontSize: '0.75rem', fontWeight: 600,

  color: priority.border, background: 'rgba(0,0,0,0.05)',

  padding: '0.2rem 0.5rem', borderRadius: '999px' }}>

  {priority.label}

</span>

{/* Catégorie */}

{task.categories && (

  <span style={{ fontSize: '0.75rem', padding: '0.2rem 0.5rem',

    borderRadius: '999px', background: task.categories.color + '33',

    color: task.categories.color, fontWeight: 600 }}>

     {task.categories.name}

  </span>

)}

{/* Date d'échéance */}

{dueLabel && (

  <span style={{ fontSize: '0.75rem', padding: '0.2rem 0.5rem',

    borderRadius: '999px', background: isOverdue ? '#FEE2E2' :

'#F1F5F9',

    color: isOverdue ? '#DC2626' : '#64748B', fontWeight: 600 }}>

     {isOverdue ? 'Δ ' : ''}{dueLabel}

  </span>

)}

</div>

</div>

);

```

}



task.categories (objet imbriqué)

Notez la syntaxe `task.categories` (avec un 's'). C'est parce que Supabase retourne l'objet lié par la FK sous le nom de la table étrangère. Lors du fetch, vous utiliserez `.select('*', categories(*))` pour récupérer aussi les données de la catégorie.

Partie D — Formulaire de création de tâche (20 min)

D.1 Créer TaskForm.jsx

Ce composant contient un formulaire pour créer une nouvelle tâche. Créez `src/components/TaskForm.jsx` :

```
// src/components/TaskForm.jsx

import { useState, useEffect } from 'react';

import { supabase } from '../lib/supabase';

export default function TaskForm({ boardId, onCreated }) {

  const [title, setTitle] = useState('');

  const [description, setDescription] = useState('');

  const [status, setStatus] = useState('todo');

  const [priority, setPriority] = useState('medium');

  const [categoryId, setCategoryId] = useState('');

  const [dueDate, setDueDate] = useState('');

  const [categories, setCategories] = useState([]);

  const [loading, setLoading] = useState(false);

  const [error, setError] = useState('');

  // Charger les catégories au montage

  useEffect(() => {

    supabase.from('categories').select('*').then(({ data }) => {

      setCategories(data || []);

    });

  }, []);

}
```

```

async function handleSubmit(e) {

  e.preventDefault();

  setError('');

  if (!title.trim()) { setError('Le titre est obligatoire. '); return; }

  setLoading(true);

  const { data: { user } } = await supabase.auth.getUser();

  const { error } = await supabase.from('tasks').insert([

    {
      title: title.trim(),

      description: description.trim() || null,

      status,

      priority,

      board_id: boardId,

      category_id: categoryId || null,

      due_date: dueDate || null,

      created_by: user.id,
    }
  ]);

  setLoading(false);

  if (error) { setError(error.message); return; }

  // Réinitialiser le formulaire

  setTitle(''); setDescription(''); setStatus('todo');

  setPriority('medium'); setCategoryId(''); setDueDate('');

  onCreate();
}

return (

  <form onSubmit={handleSubmit} style={{ background: 'white',

    padding: '1.5rem', borderRadius: '10px', marginBottom: '1.5rem',

```

```
border: '1px solid #E2E8F0' }}>
```

```
<h3 style={{ marginTop: 0, color: '#1A8C82' }}>+ Nouvelle tâche</h3>
```

```
{error} && <p style={{ color: '#DC2626' }}>{error}</p>
```

```
<input placeholder='Titre de la tâche *' value={title}
```

```
onChange={e => setTitle(e.target.value)} required
```

```
style={{ ...inputStyle, width: '100%', marginBottom: '0.75rem' }} />
```

```
<textarea placeholder='Description (optionnelle)'
```

```
value={description} onChange={e => setDescription(e.target.value)}
```

```
rows={3} style={{ ...inputStyle, width: '100%', marginBottom: '0.75rem',
```

```
resize: 'vertical' }} />
```

```
<div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr 1fr 1fr',
```

```
gap: '0.5rem', marginBottom: '0.75rem' }}>
```

```
<div>
```

```
<label style={labelStyle}>Statut</label>
```

```
<select value={status} onChange={e => setStatus(e.target.value)}
```

```
style={inputStyle}>
```

```
<option value='todo'>📋 À faire</option>
```

```
<option value='in_progress'>🌀 En cours</option>
```

```
<option value='review'>👁 Validation</option>
```

```
<option value='done'>✅ Terminée</option>
```

```
</select>
```

```
</div>
```

```
<div>
```

```
<label style={labelStyle}>Priorité</label>
```

```
<select value={priority} onChange={e => setPriority(e.target.value)}>
```

```
  style={inputStyle}>
```

```
<option value='low'> Basse</option>
```

```
<option value='medium'> Moyenne</option>
```

```
<option value='high'> Haute</option>
```

```
</select>
```

```
</div>
```

```
<div>
```

```
<label style={labelStyle}>Catégorie</label>
```

```
<select value={categoryId} onChange={e =>  
setCategoryId(e.target.value)}>
```

```
  style={inputStyle}>
```

```
<option value=''>— Aucune —</option>
```

```
{categories.map(c => (
```

```
  <option key={c.id} value={c.id}>{c.name}</option>
```

```
))}
```

```
</select>
```

```
</div>
```

```
<div>
```


Partie E — Liste de tâches & intégration dans le dashboard (20 min)

E.1 Créer le composant TaskList.jsx

Ce composant charge les tâches depuis Supabase et les affiche en utilisant TaskCard. Créez src/components/TaskList.jsx :

```
// src/components/TaskList.jsx

import { useState, useEffect } from 'react';

import { supabase } from '../lib/supabase';

import TaskCard from './TaskCard';

import TaskForm from './TaskForm';

export default function TaskList({ boardId }) {

  const [tasks, setTasks] = useState([]);

  const [loading, setLoading] = useState(true);

  async function fetchTasks() {

    setLoading(true);

    const { data, error } = await supabase

      .from('tasks')

      .select('*', categories('*')) // jointure automatique !

      .eq('board_id', boardId)

      .order('created_at', { ascending: false });

    if (!error) setTasks(data || []);

    setLoading(false);

  }

}
```

```

useEffect(() => { fetchTasks(); }, [boardId]);

async function handleDelete(taskId) {

  if (!confirm('Supprimer cette tâche ?')) return;

  await supabase.from('tasks').delete().eq('id', taskId);

  fetchTasks();

}

if (loading) return <p>Chargement des tâches...</p>;

return (

  <div>

    <TaskForm boardId={boardId} onCreate={fetchTasks} />

    <div style={{ display: 'grid',

      gridTemplateColumns: 'repeat(auto-fill, minmax(300px, 1fr))',

      gap: '0.75rem' }}>

      {tasks.map(task => (

        <TaskCard key={task.id} task={task} onDelete={handleDelete} />

      ))}

    </div>

    {tasks.length === 0 && (

      <p style={{ textAlign: 'center', color: '#94A3B8', padding: '2rem' }}>

        Aucune tâche - créez-en une ci-dessus ! 🚀

      </p>

    )}

  </div>

```

```

        </p>

    )}

</div>

);
}

```

E.2 Intégrer TaskList dans DashboardPage.jsx

Modifiez `src/pages/DashboardPage.jsx` pour intégrer la liste des tâches. Ajoutez les imports et un onglet :

```

// Ajoutez en haut de DashboardPage.jsx

import TaskList from '../components/TaskList';

// Dans le composant, ajoutez un state pour gérer l'onglet actif

const [tab, setTab] = useState('tasks');

const [boardId, setBoardId] = useState(null);

// Charger le premier board de l'utilisateur

useEffect(() => {

    supabase.from('boards').select('id').limit(1)

        .then(({ data }) => { if (data?.[0]) setBoardId(data[0].id); });

}, []);

// Dans le JSX, remplacez <main> par :

<main style={{ padding: '2rem' }}>

    {/* Navigation par onglets */}

    <div style={{ display: 'flex', gap: '0.5rem', marginBottom: '1.5rem' }}>

```

```

    {[['tasks', '📅 Tâches'], ['users', '👤 Utilisateurs']].map(([key, label])
=> (

    <button key={key} onClick={() => setTab(key)} style={{

      padding: '0.5rem 1rem', borderRadius: '6px', border: 'none',

      cursor: 'pointer',

      background: tab === key ? '#1A8C82' : '#E2E8F0',

      color: tab === key ? 'white' : '#1E293B',

      fontWeight: tab === key ? 700 : 400,

    }}>{label}</button>

    )))

</div>

{tab === 'tasks' && boardId && <TaskList boardId={boardId} />}

{tab === 'tasks' && !boardId && (

  <p style={{ color: '#94A3B8' }}>Aucun tableau trouvé. Créez-en un via SQL
Editor.</p>

)}

{tab === 'users' && (

  loading ? <p>Chargement...</p> : <UserTable users={users}
onRefresh={fetchUsers} />

)}

</main>

```

Partie F — Tests & commit final (15 min)

F.1 Scénario de test complet

4. Lancez npm run dev.
5. Connectez-vous avec votre compte.
6. Allez sur l'onglet  Tâches → les 4 tâches insérées en SQL doivent s'afficher, avec les badges de statut, priorité et catégorie.
7. Créez une nouvelle tâche via le formulaire : remplissez titre, description, priorité, catégorie et date d'échéance → cliquez Créer la tâche.
8. Vérifiez que la tâche apparaît immédiatement dans la liste.
9. Supprimez une tâche → confirmez la suppression → elle disparaît.
10. Changez d'onglet :  Utilisateurs → les profils s'affichent toujours.



Problème fréquent : 'No rows found'

Si vos tâches n'apparaissent pas, vérifiez : (1) Le board_id utilisé dans le SELECT correspond bien à un board existant. (2) Le RLS est désactivé sur la table tasks (Authentication → Policies). (3) Dans le SQL Editor, vérifiez SELECT * FROM tasks; que des lignes existent.

F.2 Commit & push

```
git add .
```

```
git commit -m "feat: add task model, TaskCard, TaskForm and TaskList"
```

```
git push origin main
```

Vercel redéploie automatiquement. Vérifiez que votre URL Vercel fonctionne en production.

Livrable de fin de séance



À déposer sur la plateforme de cours

1. URL du dépôt GitHub (doit contenir TaskCard.jsx, TaskForm.jsx, TaskList.jsx)
2. URL Vercel avec l'app fonctionnelle (connexion → dashboard → onglet Tâches)
3. Capture d'écran du dashboard affichant au moins 3 tâches avec badges de statut et priorité
4. Capture de Table Editor Supabase montrant les 3 tables (tasks, categories, boards)

Bonus — Pour aller plus loin

- Filtrer les tâches par statut (select ou boutons radio).
- Trier les tâches par priorité ou date d'échéance.
- Ajouter un badge 'En retard' (rouge clignotant) quand la date est dépassée.
- Permettre la modification d'une tâche en cliquant dessus (formulaire inline).
- Afficher un compteur de tâches par statut (ex : 3 à faire, 1 en cours...).
- Activer le Realtime Supabase sur la table tasks pour que les modifications se propagent en temps réel entre deux onglets du navigateur.